DASH-06: Variables and Filters

> **Series:** DASH | **Notebook:** 6 of 7 | **Created:** March 2026 | **Last Updated:** 04/04/2026

Overview

Variables transform a static dashboard into a dynamic, reusable tool. Instead of building separate dashboards for each service, environment, or team, a single template dashboard with variables can serve everyone. This notebook covers variable types in Dynatrace dashboards, filter propagation across tiles, variable-driven DQL queries, entity selector patterns, and strategies for building template dashboards that work across environments.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled
Permissions	`storage:logs:read`, `storage:metrics:read`, `storage:spans:read`, `storage:entities:read`
Dashboard Access	`document:documents:write` for creating dashboards with variables
Prior Reading	DASH-01 through DASH-05

1. Variable Types

Dynatrace dashboards support several variable types, each suited for different filtering needs.

Variable Type	Description	Use Case
Entity selector	Dropdown of Dynatrace entities (hosts, services, etc.)	Filter by specific service or host
String	Free-text or predefined string values	Environment names, namespaces, log levels
Query-based	Values populated from a DQL query	Dynamic lists based on actual data
Time range	Dashboard-wide time selector	Override default time range

Variable Naming Conventions

Convention	Example	Notes
Descriptive name	`$service`, `$environment`	Immediately clear what it filters

Convention	Example	Notes
Prefix by type	`$entity_host`, `$str_namespace`	Useful in complex dashboards
Lowercase with underscores	`$k8s_namespace`	Consistent, readable in queries

> **Important:** Variable names are case-sensitive. Use consistent casing across all tiles that reference the same variable.

2. Entity Selector Variables

Entity selector variables are the most common variable type. They present a searchable dropdown of Dynatrace monitored entities.

Common Entity Selectors

Variable	Entity Type	Dashboard Use
`$host`	`dt.entity.host`	Filter infrastructure metrics to a specific host
`$service`	`dt.entity.service`	Filter spans and service metrics
`$process_group`	`dt.entity.process_group`	Filter process-level metrics
`$k8s_cluster`	`dt.entity.kubernetes_cluster`	Filter K8s workloads

Querying Available Entities for Variable Population

```
// List all services — useful for populating a service variable
fetch dt.entity.service
| fieldsKeep id, entity.name
| sort entity.name asc

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes SERVICE
// | fieldsKeep id, name
// | sort name asc
```

Querying Hosts for Variable Population

```
// List all hosts — useful for populating a host variable
fetch dt.entity.host
| fieldsKeep id, entity.name
| sort entity.name asc

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes HOST
// | fieldsKeep id, name
// | sort name asc
```

3. String Variables

String variables accept free-text or predefined values. They are ideal for filtering on attributes like environment, namespace, or log level.

Discovering Available Values

Before creating a string variable with predefined options, query the data to discover what values exist.

```
// Discover Kubernetes namespaces for variable options
fetch logs, from:-1h
| filter isNotNull(k8s.namespace.name)
| summarize log_count = count(), by:{k8s.namespace.name}
| sort log_count desc
```

```
// Discover log levels for variable options
fetch logs, from:-1h
| summarize log_count = count(), by:{loglevel}
| sort log_count desc
```

Discovering Available Database Systems

```
// Discover database systems for variable options
fetch spans, from:-1h
| filter span.kind == "client" and isNotNull(db.system)
| summarize query_count = count(), by:{db.system}
| sort query_count desc
```

4. Variable-Driven DQL Queries

Once variables are defined on a dashboard, reference them in tile DQL queries using the `$variable_name` syntax.

Patterns for Using Variables in DQL

Pattern	DQL Example	Notes
Entity filter	`filter dt.entity.host == $host`	Entity selector variable
String filter	`filter k8s.namespace.name == $namespace`	String variable
Pattern match	`filter k8s.namespace.name ~ $namespace_pattern`	Wildcard string variable
Multi-select	`filter in(loglevel, $log_levels)`	Multi-value string variable

Example: Service Latency Filtered by Variable

In a dashboard tile, this query would use the `$service` variable. In a notebook, we use a concrete value to demonstrate the pattern.

```
// Service latency filtered by entity — dashboard would use $service variable
// In a dashboard tile: filter dt.entity.service == $service
fetch spans, from:-1h
| filter span.kind == "server"
| filter isNotNull(dt.entity.service)
| makeTimeseries p50_ms = percentile(duration, 50) / 1000000, p95_ms =
percentile(duration, 95) / 1000000, interval:5m, by:{dt.entity.service}
```

Example: Log Analysis with Namespace and Level Variables

```
// Log analysis — dashboard would use $namespace and $loglevel variables
// In a dashboard tile: filter k8s.namespace.name == $namespace and loglevel
== $loglevel
fetch logs, from:-1h
| filter isNotNull(k8s.namespace.name)
| filterOut loglevel == "NONE"
| makeTimeseries log_count = count(), interval:5m, by:{k8s.namespace.name,
loglevel}
```

Example: Metrics with Host Variable

```
// CPU and memory for selected host — dashboard would use $host variable
// In a dashboard tile: filter:dt.entity.host == $host
timeseries avg_cpu = avg(dt.host.cpu.usage), from:-2h, by:{dt.entity.host}
| fieldsAdd avg_cpu_val = arrayAvg(avg_cpu)
| sort avg_cpu_val desc
| limit 5
```

5. Filter Propagation

Filter propagation ensures that when a user selects a variable value, all relevant tiles update simultaneously.

How Filter Propagation Works

1. User selects a value from the variable dropdown (e.g., selects a specific service)
2. All tiles that reference `$service` in their query re-execute with the new filter
3. Tiles that do not reference the variable remain unchanged

Best Practices for Filter Propagation

Practice	Description
Use the same variable name in all related tiles	Ensures consistent filtering across the dashboard
Document which tiles respond to which variables	Add a markdown tile listing variable-tile mappings
Provide an "All" option	Let users remove the filter to see aggregate data
Test with extreme values	Select the busiest and quietest entities to verify layout holds
Chain variables	Use one variable's value to filter another variable's options

Variable Chaining Pattern

For hierarchical filtering (e.g., select a cluster, then see only namespaces in that cluster), use query-based variables where the second variable's query references the first.

```
// Namespaces filtered by cluster — would be a query-based variable
// In variable config: references $cluster variable
fetch logs, from:-1h
| filter isNotNull(k8s.namespace.name) and isNotNull(k8s.cluster.name)
| summarize log_count = count(), by:{k8s.cluster.name, k8s.namespace.name}
| sort k8s.cluster.name asc, log_count desc
```

6. Template Dashboard Patterns

Template dashboards use variables so aggressively that a single dashboard serves multiple teams or environments.

Pattern 1: Multi-Environment Dashboard

A single dashboard with an `$environment` variable (prod, staging, dev) that filters all tiles.

Variable	Values	Applied To
`$environment`	prod, staging, dev	Namespace filter, host group filter
`$service`	(entity selector)	Service-specific tiles
`$time_range`	15m, 1h, 6h, 24h	All tiles

Pattern 2: Team-Owned Service Dashboard

Each team uses the same template but selects their services.

Variable	Source	Notes
`$team_services`	Query-based: services tagged with team name	Multi-select entity variable
`$severity`	String: ERROR, WARN, INFO	Log level filter

Discovering Tags for Team-Based Variables

```
// Discover service tags — useful for building team-based variables
fetch dt.entity.service
| expand tags
| summarize service_count = count(), by:{tags}
| sort service_count desc
| limit 20
```

Pattern 3: Golden Signals Template

A template dashboard that displays the four golden signals (latency, traffic, errors, saturation) for any selected service.

Section	Query Pattern	Chart Type
Latency	`percentile(duration, 95)` filtered by `$service`	Line chart
Traffic	`count()` of server spans filtered by `$service`	Line chart
Errors	`countIf(otel.status_code == "ERROR")` filtered by `$service`	Line chart
Saturation	CPU/memory metrics for the service's host	Line chart

This pattern is especially powerful because it works for every service in the environment — engineers just change the variable.

7. Summary and Next Steps

In this notebook you learned:

- The four variable types available in Dynatrace dashboards
- How to create entity selector and string variables
- DQL patterns for referencing variables in tile queries
- Filter propagation mechanics and best practices
- Template dashboard patterns: multi-environment, team-owned, and golden signals

Next: In **DASH-07: Sharing and Reporting**, we cover dashboard permissions, sharing with teams, scheduled reports via Workflows, dashboard-as-code, and version control patterns.
